



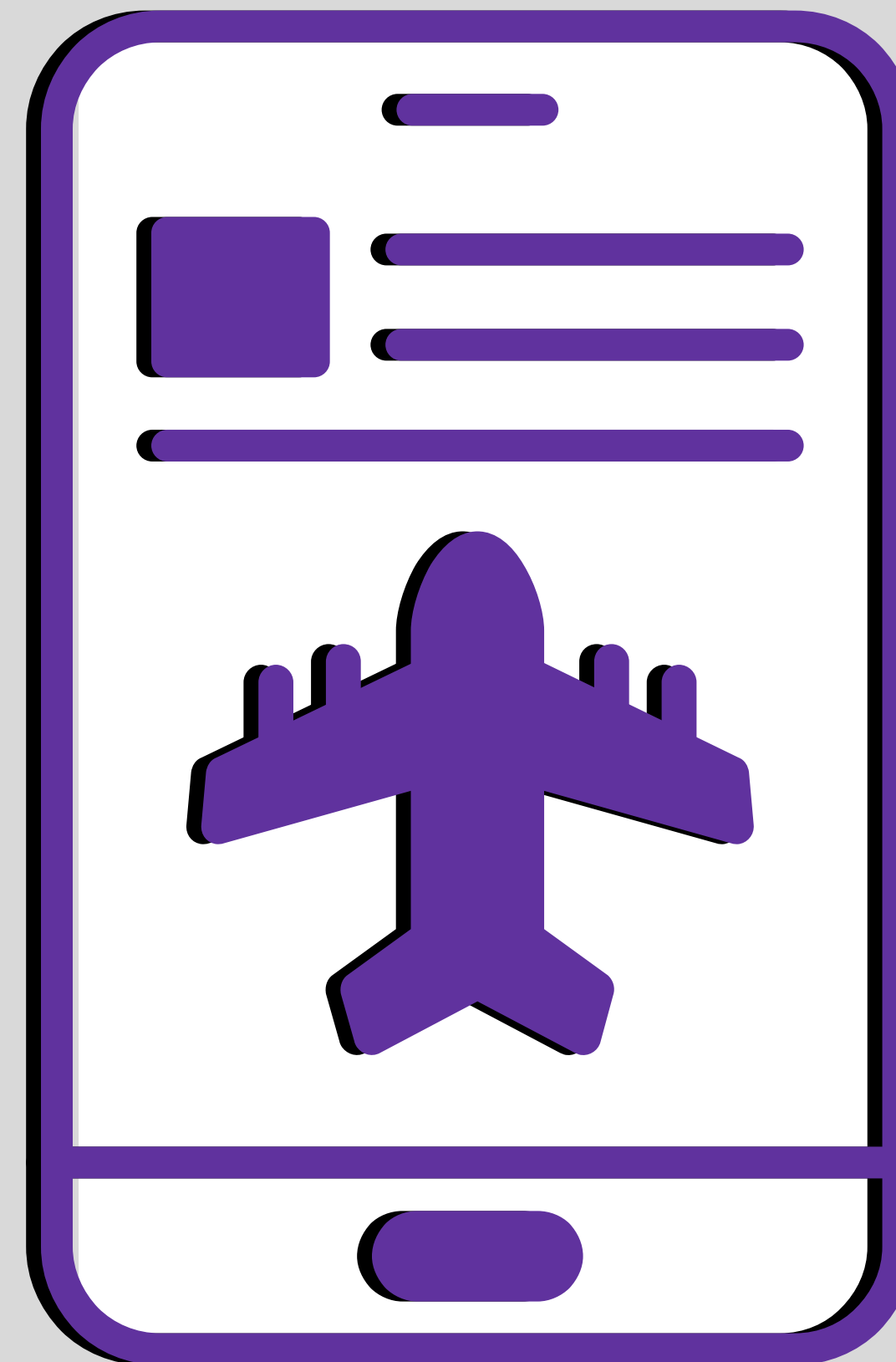
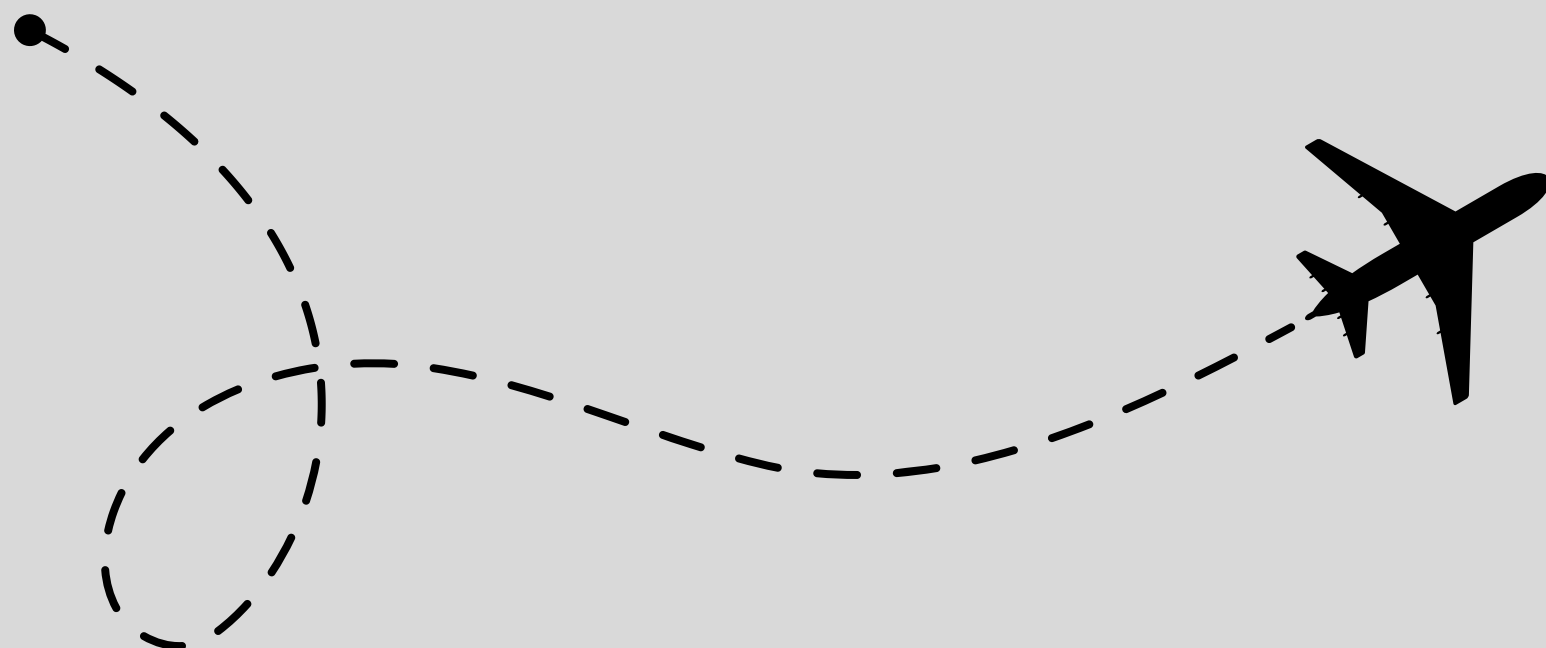
Produto e desenvolvimento





Ideia do produto

O OwnMyWay é um aplicativo de assistência de viagem feito para facilitar o processo de criação de rotas para viagem baseado nos seus gostos, hobbies e personalidade, transformando o ato de viajar em um experiência com a sua cara.

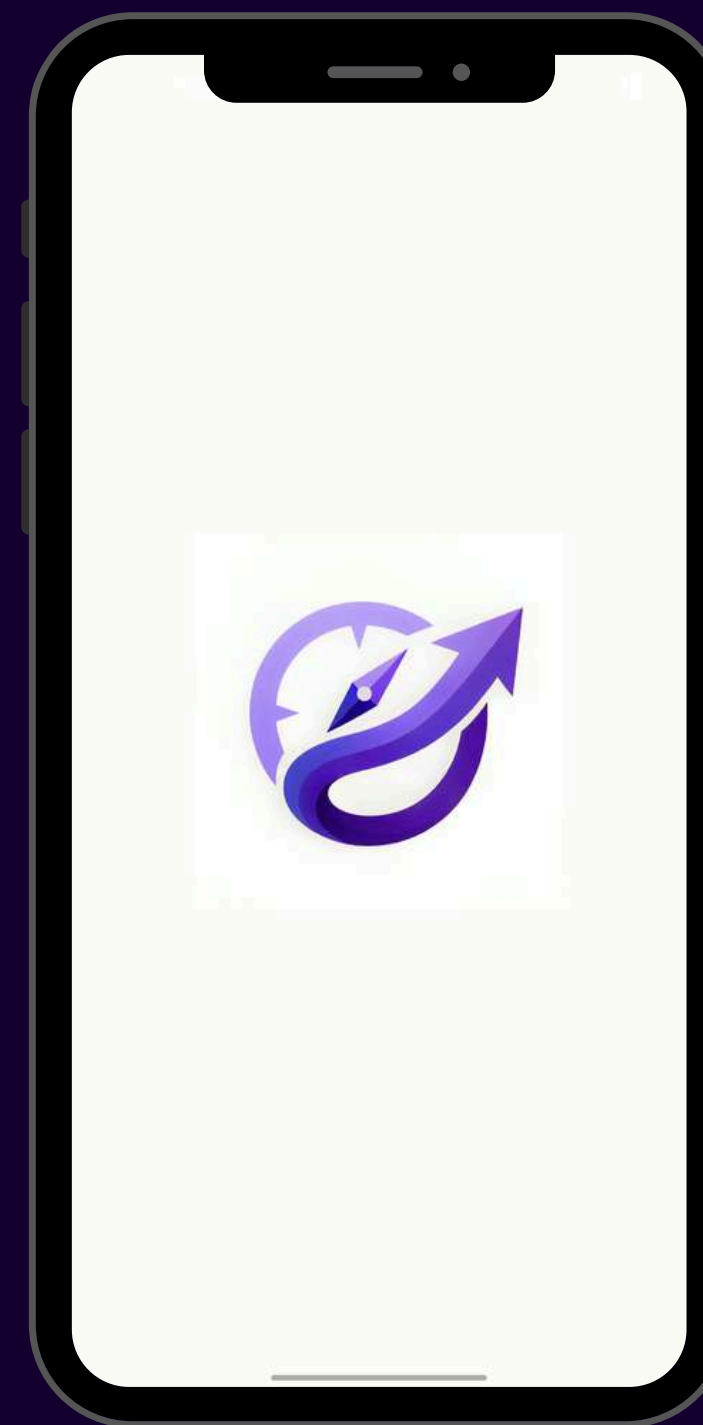




Tecnologias aplicadas

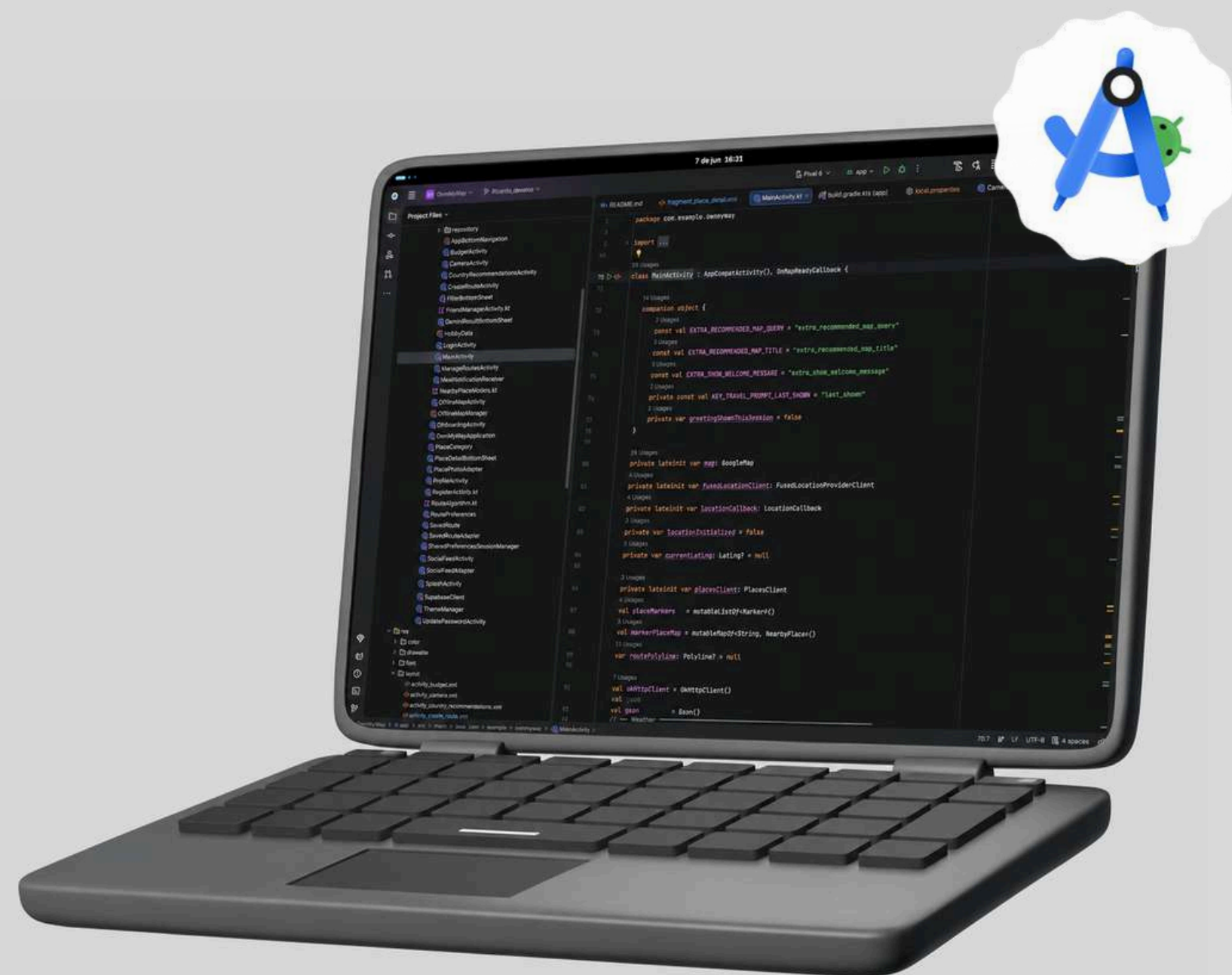
Com relação à linguagem, nós utilizamos o Kotlin para o desenvolvimento nativo android.

Por ser a nossa primeira experiência com desenvolvimento mobile no geral, decidimos focar em uma plataforma ao invés de desenvolvimento híbrido e o Kotlin foi escolhido por ser o padrão no desenvolvimento android e grande parte das APIs android foram feitas nativamente para o Kotlin, também o código é bem mais fácil de entender e manter do que o Java, que tende a ser muito verboso.





O android studio como IDE por conta da fácil emulação de dispositivos e quantidade gigantesca de recursos. É um pouco difícil de lidar no início justamente por conta da quantidade de opções, mas quando dominada é uma ferramenta incrível para debugging.





Utilizamos o Supabase como o host do nosso banco de dados e também como sistema de autenticação.

Com relação a serviços utilizamos praticamente todo o ecossistema da API do Maps, desde a tecnologia de criação de rotas até informações quanto ao clima. Esse serviço é a base de praticamente toda a aplicação.

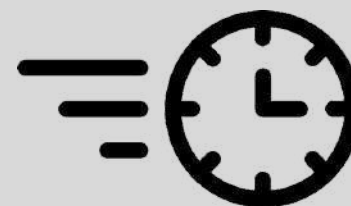


Outras tecnologias menores foi o Retrofit, biblioteca utilizada para lidar com requisições na internet, ou seja, permitir o uso de APIs. Também, o CameraX que é outra biblioteca que permite o desenvolvimento de aplicações que interagem com a câmera do celular.

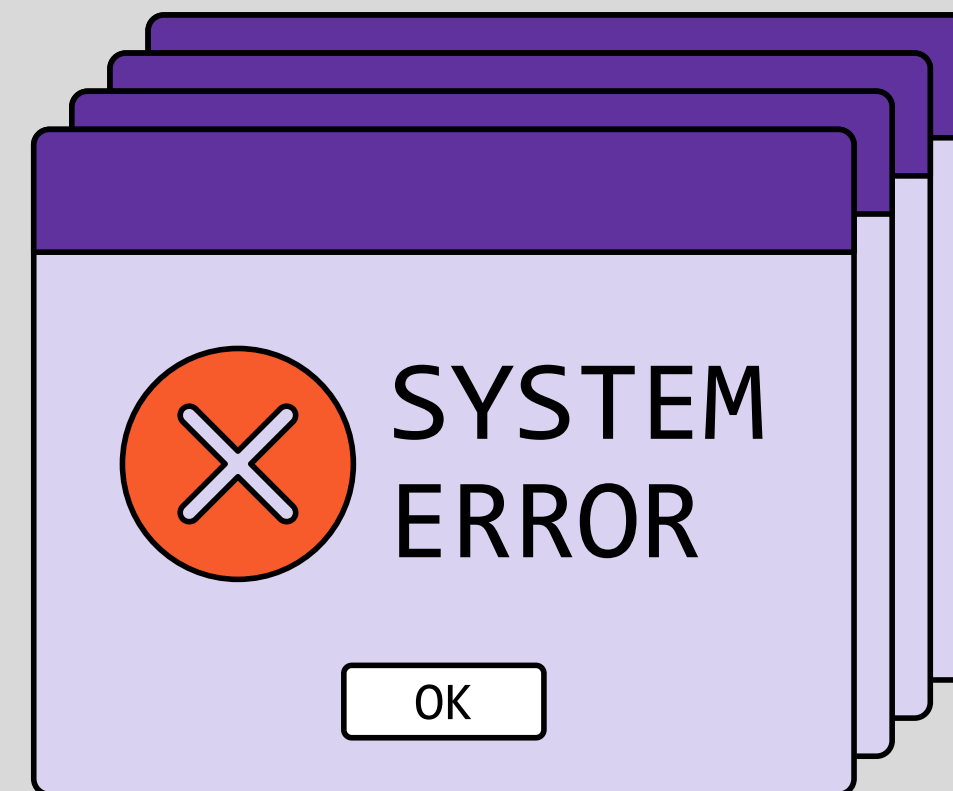


Dificuldades no desenvolvimento

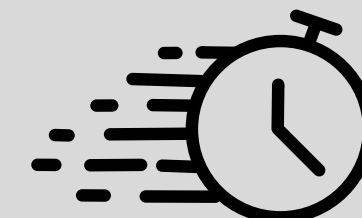




Grande parte das dificuldades veio da falta de experiência com as ferramentas utilizadas. O desenvolvimento em Kotlin, o uso da IDE e as sincronizações do Gradle foram experiências novas para a equipe, gerando erros quando alguma etapa não era realizada corretamente.



Embora o Supabase também fosse uma tecnologia nova, ele foi mais simples de utilizar. Por isso, na Sprint 1 mantivemos uma quantidade normal de tarefas, mas com atividades mais simples, voltadas à construção das bases do sistema.





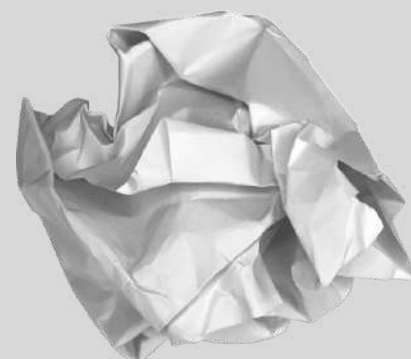
Com isso, todos trabalharam na estrutura do sistema e compreenderam melhor seus recursos. Já a Sprint 2 compensou com tarefas mais numerosas e complexas. Outra dificuldade foram ambiguidades nas descrições das sprints, que geraram interpretações diferentes e resultados fora do esperado.





Features planejadas, mas não implementadas:

- **Rota rápida:** inicialmente, planejamos um botão capaz de criar uma rota automaticamente, usando as informações já salvas do usuário. Porém, a funcionalidade foi considerada redundante, pois a criação de rota normal já mantém esses dados preenchidos e permite gerar uma rota com poucos cliques.
- **O que seus amigos estão fazendo:** a proposta era criar uma interação social entre usuários, com notificações quando amigos salvassem destinos, iniciassem uma viagem ou chegassem a determinados lugares. A ideia não foi concluída dentro do desenvolvimento final.



- **Assistente com visão computacional:** a funcionalidade previa o uso de inteligência artificial para auxiliar o usuário a partir de imagens ou reconhecimento visual. No entanto, não foi possível avançar por falta de uma API adequada para implementação.
- **Resultado geral:** apesar dessas features não finalizadas, aproximadamente 94% do produto total foi desenvolvido.

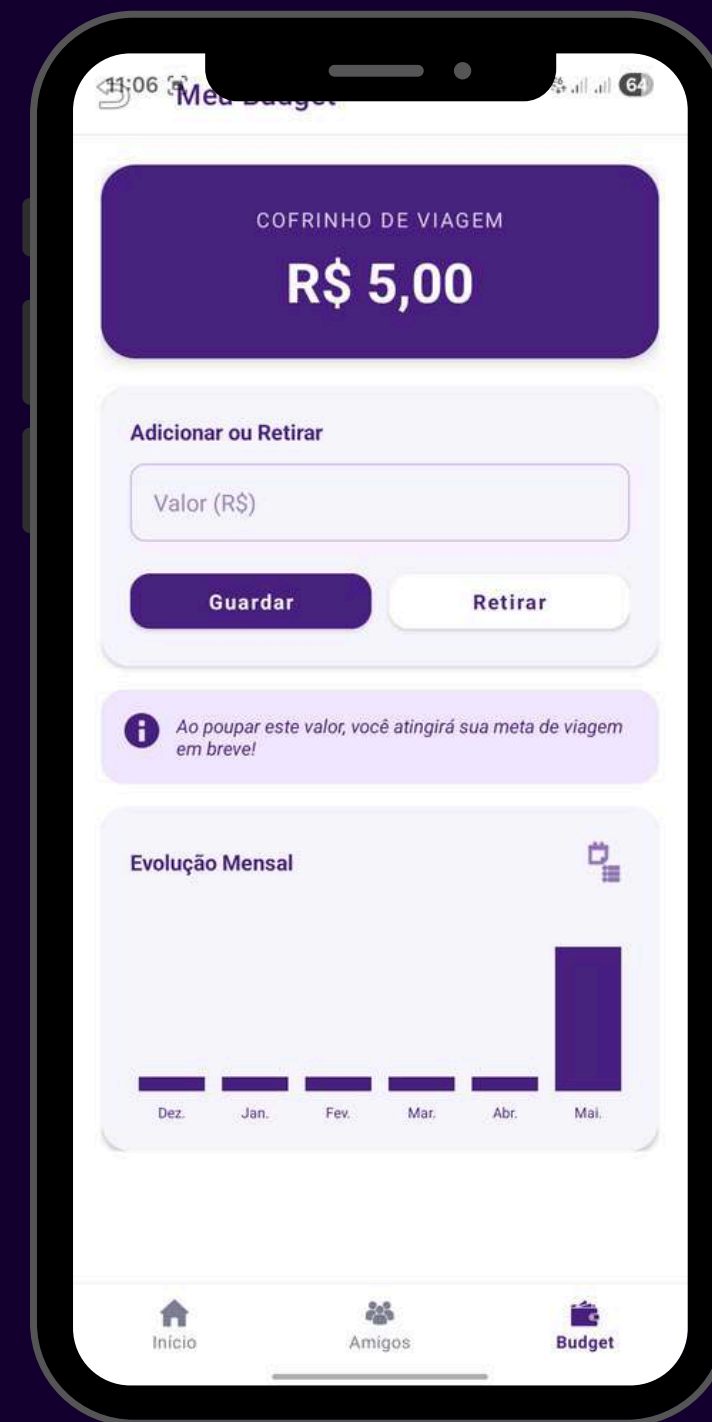
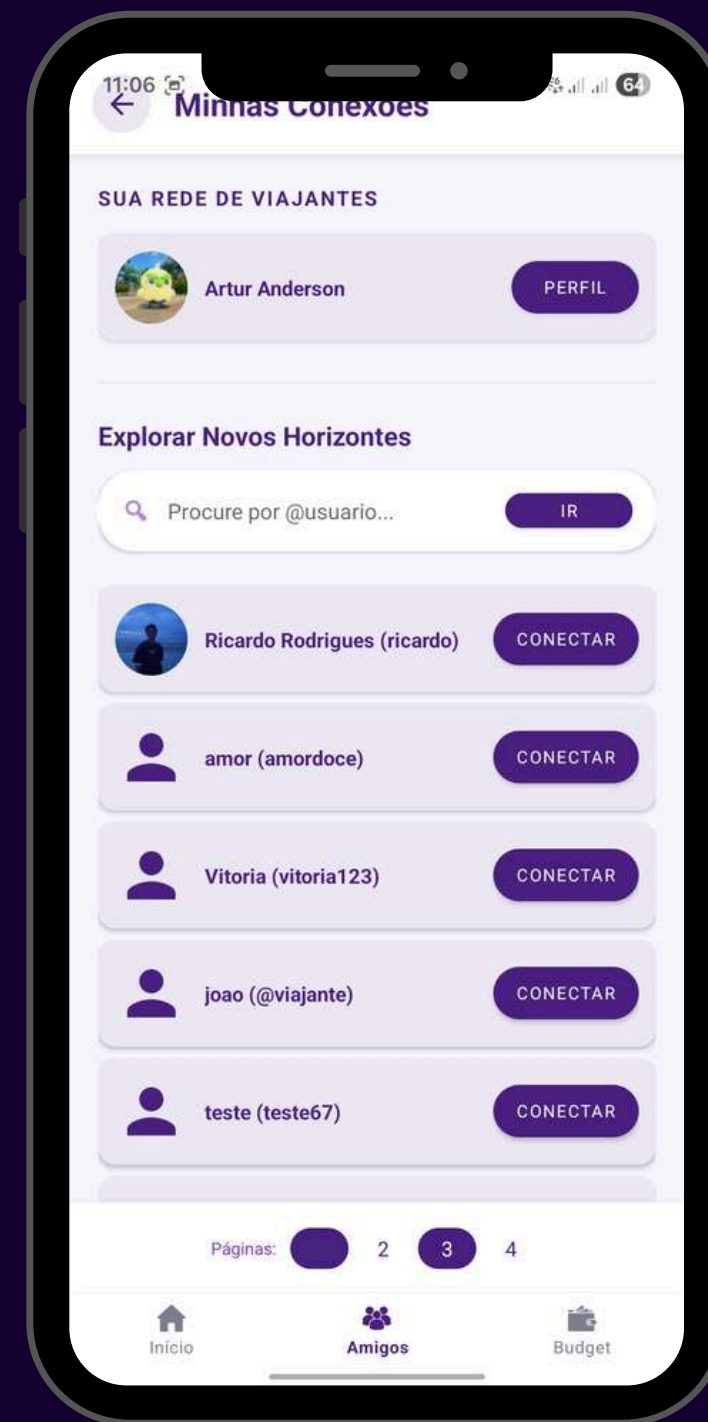


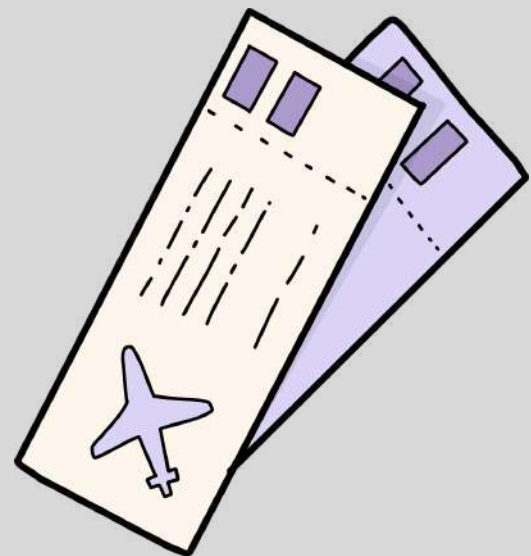


Como vocês avaliam o produto entregue? A proposta de valor foi atingida?

O produto entregue, ainda que falho em detalhes limitantes como o algoritmo que precisa de ainda mais refinamento, detalhes de UI/UX e mais interatividade com amigos, entrega a principal proposta dele que é oferecer conveniência a criação de rotas, que são feitas com base nos gostos do usuário assim como projetado.

Por isso, a proposta de valor do nosso produto que era facilitar o processo da viagem foi concluída mesmo que não dá maneira perfeita e ideal, abrindo espaço para melhora contínua.





O produto está pronto para uso?



Concluindo, o produto está pronto para uso, pois as principais features que fazem parte diretamente do valor do produto foram desenvolvidas.





OS DOZE FATORES

Contributors

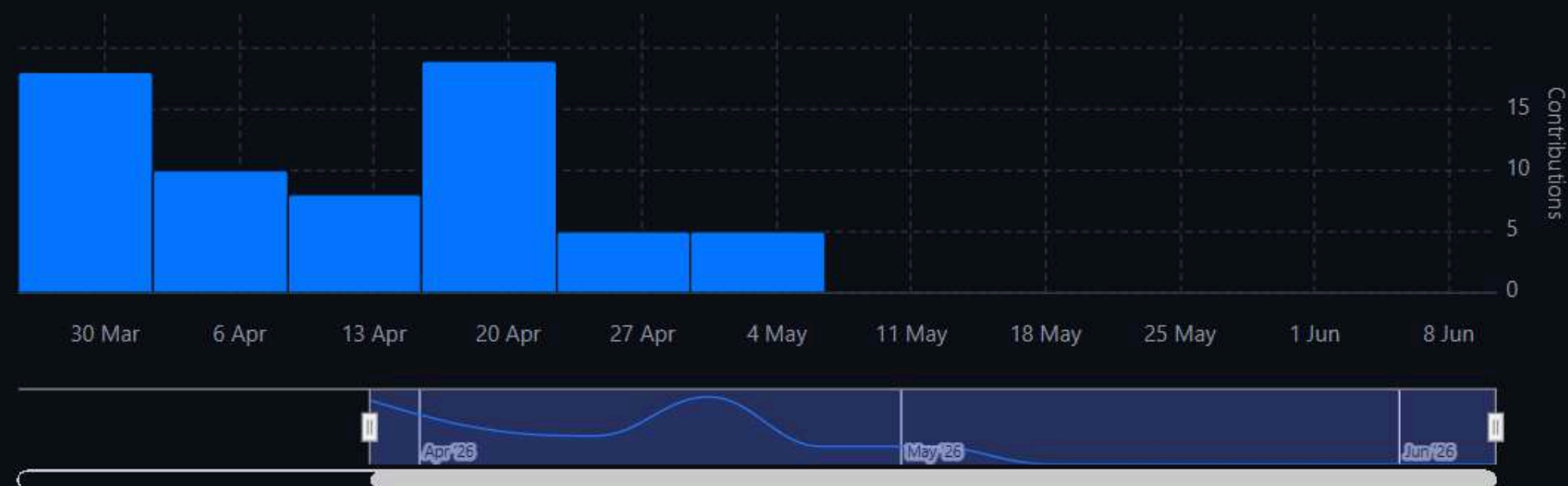
Contributions per week to main, excluding merge commits

Period: Last 3 months

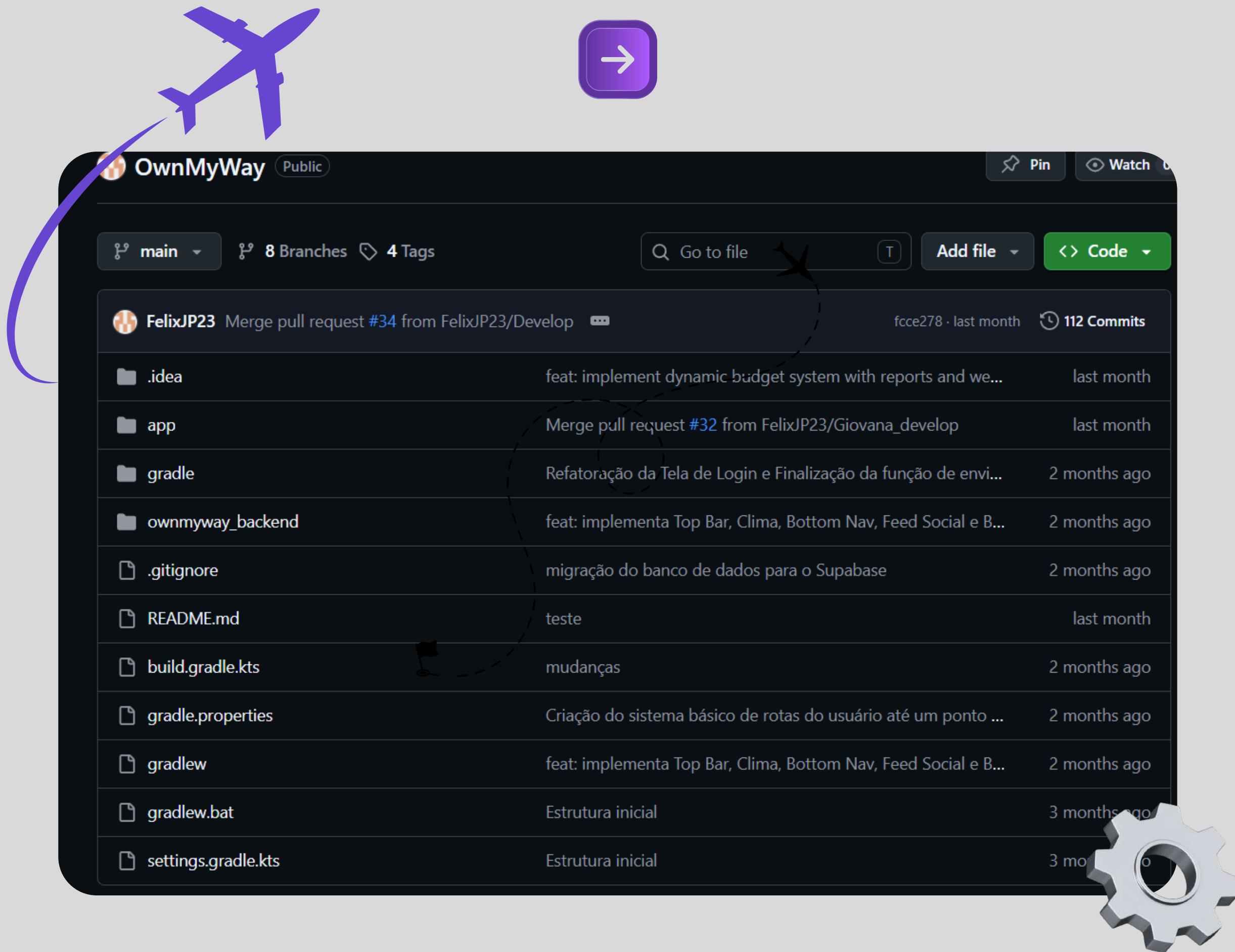
Contributions: Commits

Commits over time

Weekly from 7 de mar. de 2026 to 6 de jun. de 2026



Base de código



OwnMyWay Public

main

8 Branches 4 Tags

Go to file

T

Add file

Code



FelixJP23 Merge pull request #34 from FelixJP23/Develop

fcce278 · last month 112 Commits

.idea

feat: implement dynamic budget system with reports and we... last month

app

Merge pull request #32 from FelixJP23/Giovana_develop last month

gradle

Refatoração da Tela de Login e Finalização da função de envi... 2 months ago

ownmyway_backend

feat: implementa Top Bar, Clima, Bottom Nav, Feed Social e B... 2 months ago

.gitignore

migração do banco de dados para o Supabase 2 months ago

README.md

teste last month

build.gradle.kts

mudanças 2 months ago

gradle.properties

Criação do sistema básico de rotas do usuário até um ponto ... 2 months ago

gradlew

feat: implementa Top Bar, Clima, Bottom Nav, Feed Social e B... 2 months ago

gradlew.bat

Estrutura inicial 3 months ago

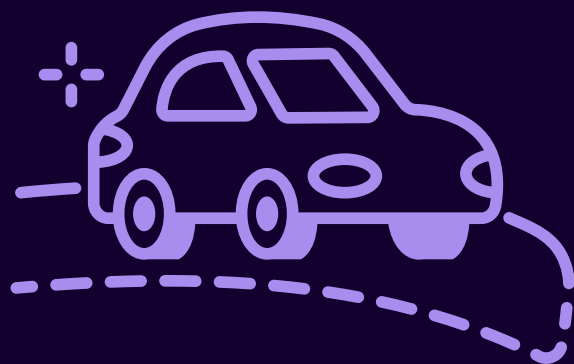
settings.gradle.kts

Estrutura inicial 3 months ago



Dependências

Todas as nossas dependências estão isoladas no build gradle do projeto.



```
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.androidx.junit)  
    implementation(libs.androidx.espresso.core)  
  
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")  
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.3")  
  
    implementation("com.google.android.gms:play-services-maps:19.0.0")  
    implementation("com.google.android.gms:play-services-location:21.3.0")  
  
    implementation(platform("io.github.jan-tennert.supabase:bom:3.1.4"))  
    implementation("io.github.jan-tennert.supabase:postgrest-kt")  
    implementation("io.github.jan-tennert.supabase:auth-kt")  
    implementation("io.ktor:ktor-client-android:3.0.1")  
    implementation("io.github.jan-tennert.supabase:storage-kt")  
  
    implementation("androidx.camera:camera-core:1.4.0")  
    implementation("androidx.camera:camera-camera2:1.4.0")  
    implementation("androidx.camera:camera-lifecycle:1.4.0")  
    implementation("androidx.camera:camera-view:1.4.0")  
}
```




logs→Manter Logs como sequências de eventos.

```
----- PROCESS STARTED (12340) for package com.example.ownmyway -----
2026-06-07 14:40:12.765 12340-12340 xample.ownmyway com.example.ownmyway I Using CollectorTypeCMC GC.
2026-06-07 14:40:12.796 12340-12340 natieloader com.example.ownmyway D Load libframework-connectivity-tiramisu-jni.so using APEX ns com.android.tethering for caller /apex/com.android.tethering/javalib/framework-connectivity-t.jar: ok
2026-06-07 14:40:12.803 12340-12340 ActivityThread com.example.ownmyway I Current dynamic renderEngine type=0
2026-06-07 14:40:12.840 12340-30996 MiviDownscaleImpl com.example.ownmyway D set override inverted scale : 1.0
2026-06-07 14:40:12.844 12340-30996 AppScoutStateMachine com.example.ownmyway D 12340-ScoutStateMachinecreated
2026-06-07 14:40:12.857 12340-12340 RENDER_TURBO com.example.ownmyway I onActivityThreadCreate not in white list, pkg=com.example.ownmyway, type=degas
2026-06-07 14:40:12.879 12340-12340 ActivityThread com.example.ownmyway D setEmbeddedParam packageName=com.example.ownmyway processName=com.example.ownmyway isEmbedded=false isIsolated=false
2026-06-07 14:40:12.881 12340-12340 ApplicationLoaders com.example.ownmyway D Returning zygot-cached class loader: /system/framework/org.apache.http.legacy.jar
2026-06-07 14:40:13.137 12340-12340 natieloader com.example.ownmyway D Configuring clns-i0 for other apk /data/app/~-RWmrkVBep5t3JxqBjaH5HA==/com.example.ownmyway-08Aumhw298aeAyWbgMrsHw==/base.apk. target_sdk_version=30, uses_libraries=, library_path=/data
2026-06-07 14:40:13.141 12340-12340 xample.ownmyway com.example.ownmyway I AssetManager2(0xb400007641a2a678) locale list changing from [] to [en-US]
2026-06-07 14:40:13.143 12340-12340 MiviPreloadClassImpl com.example.ownmyway E invoke error.
2026-06-07 14:40:13.144 12340-31007 MiviMonitorThread com.example.ownmyway D Shared singleton Thread for stability monitor start
2026-06-07 14:40:13.145 12340-12340 HWUI com.example.ownmyway D [HWUI] Flag vulkan_enhance_pipeline_cache enabled
2026-06-07 14:40:13.145 12340-12340 HWUI com.example.ownmyway D [HWUI] EnhanceVkPipelineCache is disabled
2026-06-07 14:40:13.145 12340-12340 HWUI com.example.ownmyway D [HWUI] Flag ddre enabled
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V Currently set values for:
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V angle_gl_driver_selection_pkgs=[com.android.angle, com.linecorp.b612.android, com.camnmobile.snow, com.google.android.apps.tachyon]
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V angle_gl_driver_selection_values=[angle, native, native, native]
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V com.example.ownmyway is not listed in per-application setting
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V ANGLE allowlist from config:
2026-06-07 14:40:13.146 12340-12340 GraphicsEnvironment com.example.ownmyway V com.example.ownmyway is not listed in ANGLE allowlist or settings, returning default
2026-06-07 14:40:13.147 12340-12340 GraphicsEnvironment com.example.ownmyway V App is not on the allowlist for updatable production driver.
2026-06-07 14:40:13.148 12340-31007 ActivityThread com.example.ownmyway I TrafficStats init done
```

Usamos o Logcat e logs no código para detectar erros





CONFIG→

Chaves de sensíveis estão todas isoladas no local.properties

```
11  
12 MAPS_API_KEY=  
13 SUPABASE_URL=  
14 SUPABASE_KEY=  
15 GEMINI_API_KEY=
```



BACKING Services

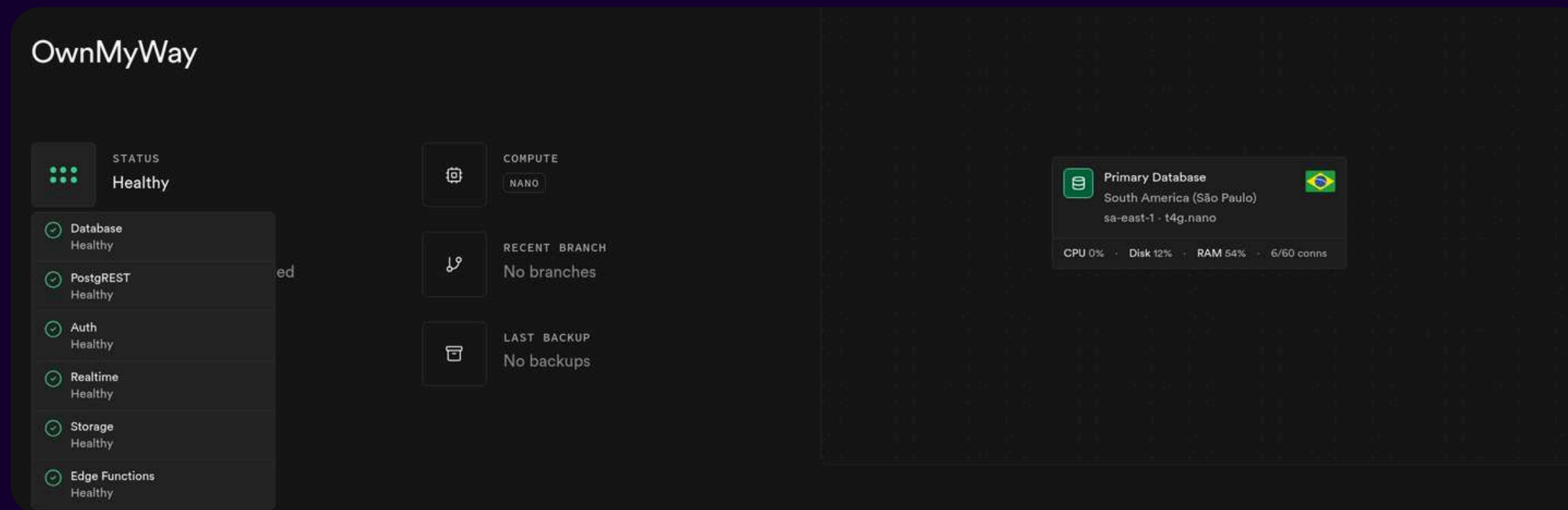
O uso do `local.properties` e do `build gradle` nas fotos anteriores já demonstram essa propriedade da nossa aplicação, em que os nossos serviços eles estão sendo acessados através de chaves e URL.

```
// USANDO URL DA API COMO EXEMPLO.
1 Usage
private fun fetchWeather(latLng: LatLng) {
    CoroutineScope(context = Dispatchers.IO).launch {
        try {
            val url = "https://api.open-meteo.com/v1/forecast" +
                "?latitude=${latLng.latitude}&longitude=${latLng.longitude}" +
                "&current=temperature_2m,weather_code"
            val body = okHttpClient.newCall(request = okhttp3.Request.Builder().url(url).build()).execute().body?.string() ?: ""
            val current = JSONObject(body).getJSONObject(name = "current")
            val temp = current.getDouble(name = "temperature_2m")
            val code = current.getInt(name = "weather_code")
            withContext(Dispatchers.Main) {
                currentWeatherCode = code
                tvTemperature.text = "${temp.toInt()}°C"
                updateWeatherUI(code)
            }
        } catch (e: Exception) {
            Log.e(tag = "MainActivity", msg = "Weather Error", tr = e)
            withContext(Dispatchers.Main) { tvWeatherCondition.text = "Erro" }
        }
    }
}
```



Processes

Toda a informação que precisamos guardar está guardada em um serviço backend, no nosso caso é o supabase. Por isso, podemos desinstalar e instalar o aplicativo e fazer alterações sem perder nenhuma informação.





Critérios sobre o produto





Existe uma versão mínima do produto (MVP) implantada?

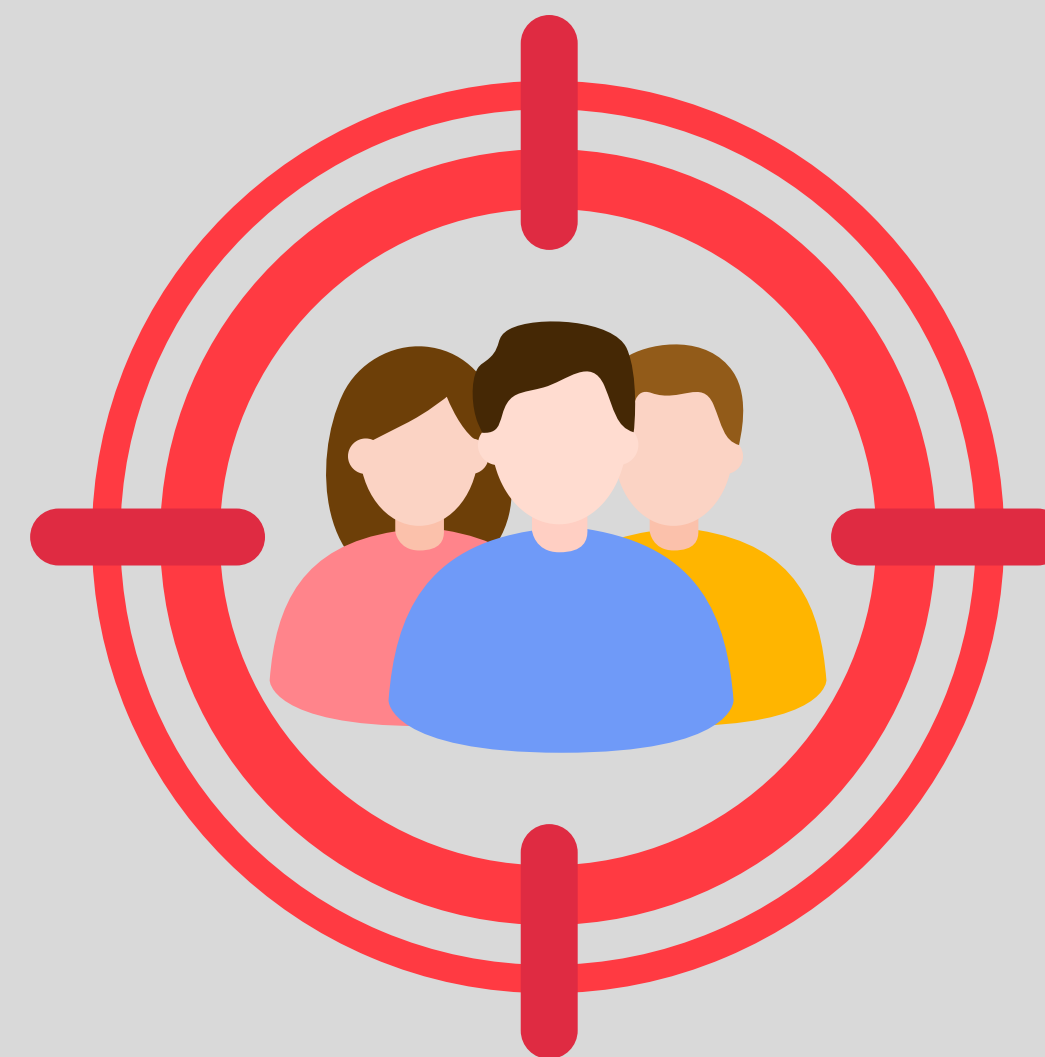
O MVP está funcional e integrado ao Supabase, cobrindo o fluxo principal do usuário: triagem de preferências e orçamento, mapa, gerenciamento de rotas, estimativas de custo por localização e perfil com sessão persistente.





Elicitação e validação dos requisitos com o público-alvo

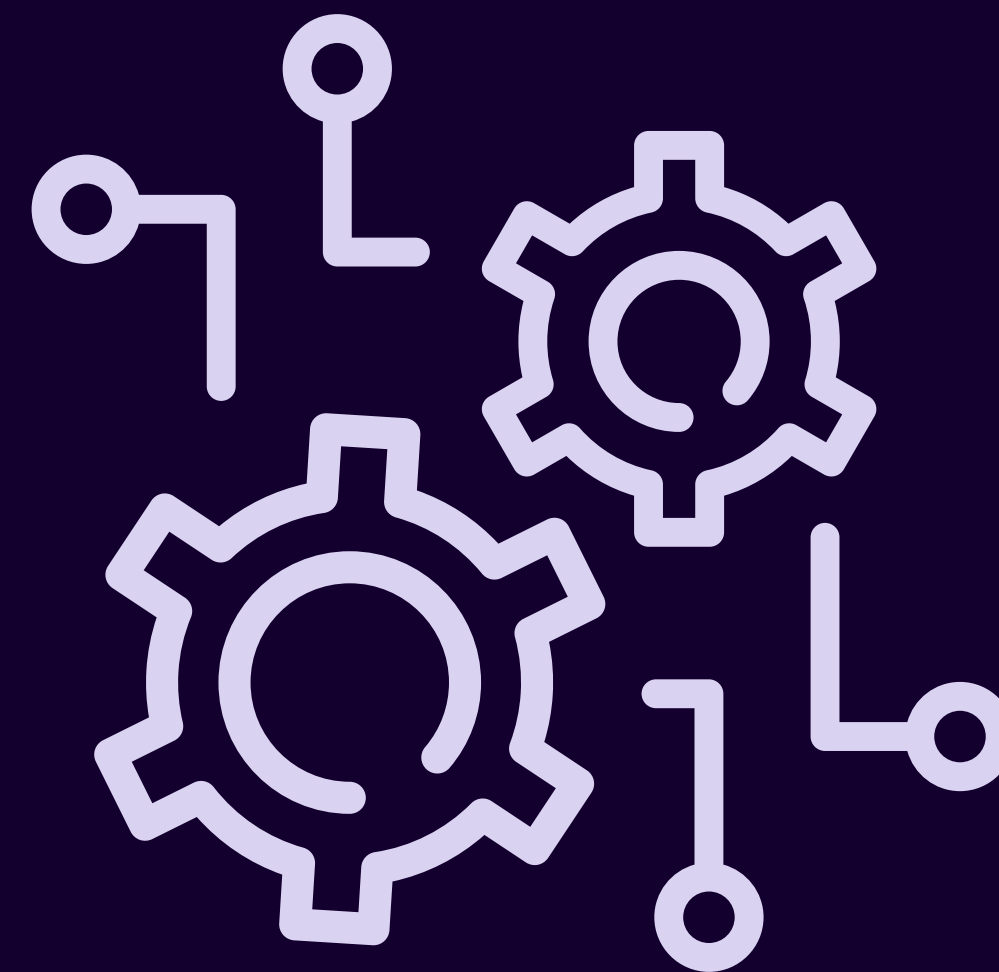
A validação do produto considera os hábitos digitais da Geração Z e dos Millennials, transformando o conceito de WhyCation em funcionalidades práticas, como tags de interesse e compartilhamento social.





Funcionalidades essenciais foram implementadas?

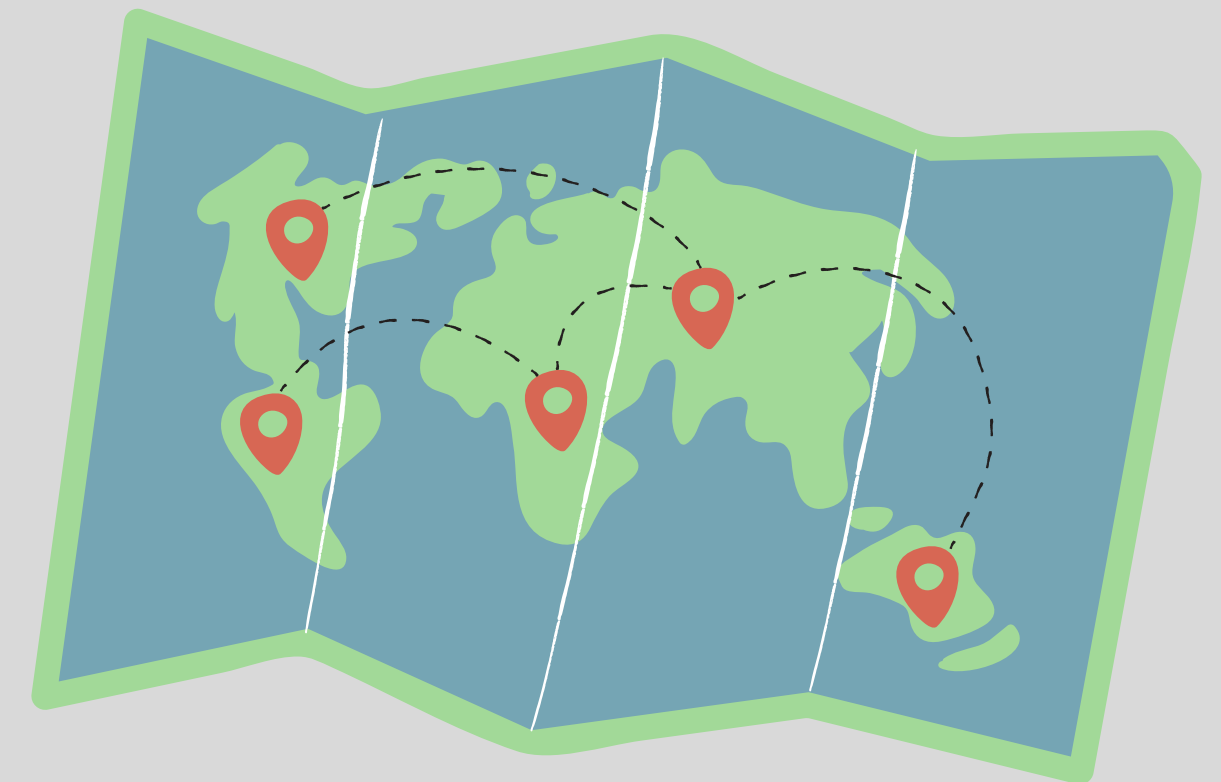
Sim. O escopo essencial do produto inclui mapa interativo, menu de acesso rápido, estimativa de custos por local, gerenciamento de rotas e perfil do usuário com persistência de dados local e remota.





O valor é facilmente perceptível aos usuários? Eles identificam rapidamente como o produto resolve seus problemas?

O valor do Own My Way está em oferecer rotas personalizadas de forma rápida, unindo orçamento e estilo de vida do usuário em um roteiro centralizado, diferente das ferramentas tradicionais focadas apenas em logística.

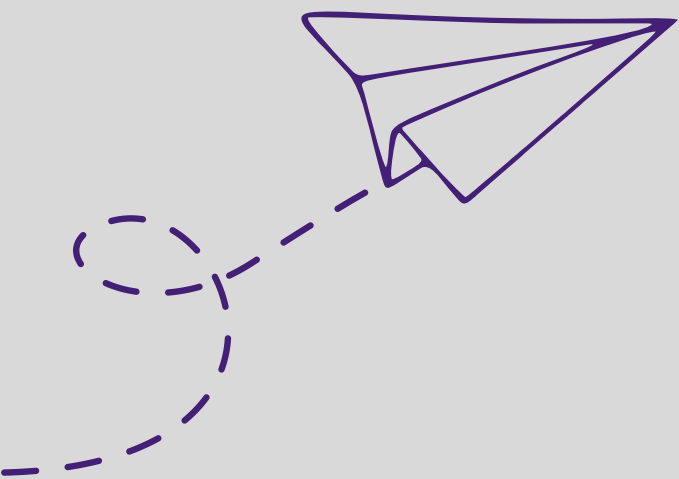




Houve um olhar criterioso para os concorrentes? Ficou clara a análise realizada?

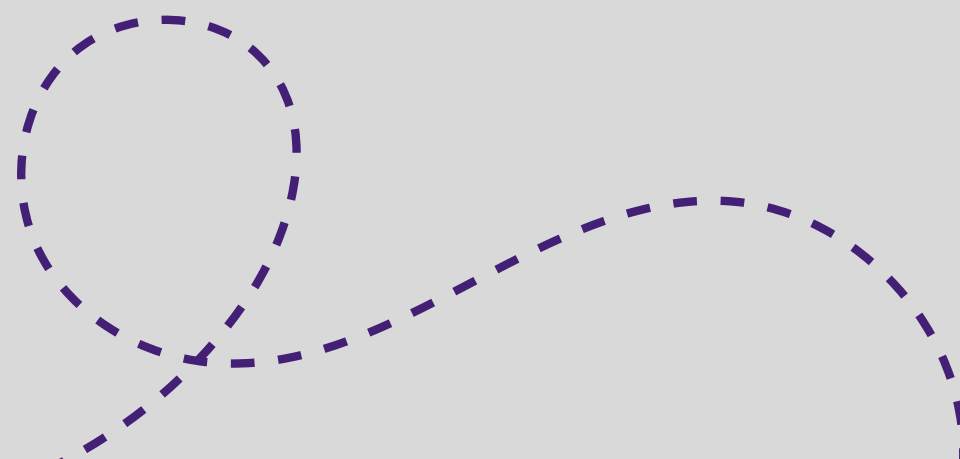
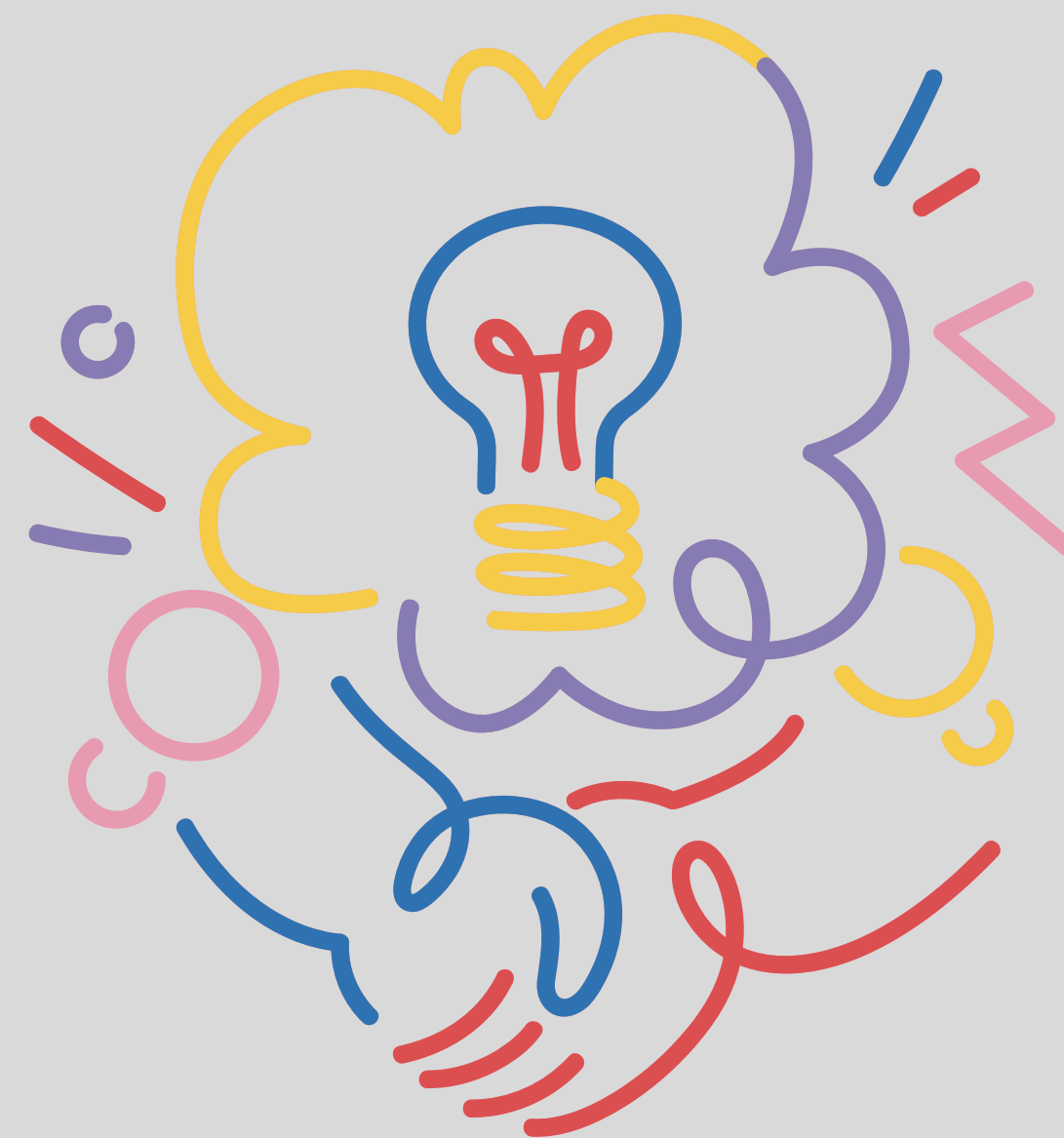
A análise competitiva mostrou que o Own My Way se diferencia por integrar serviços consolidados, como Booking.com e Uber, oferecendo conveniência ao usuário em vez de competir diretamente com essas plataformas.





Houve criatividade e/ou elementos inovadores?

O diferencial inovador está em unir planejamento de viagem com rede social, permitindo adicionar amigos, compartilhar perfis e listas de desejos, tornando a organização da viagem mais interativa e colaborativa.

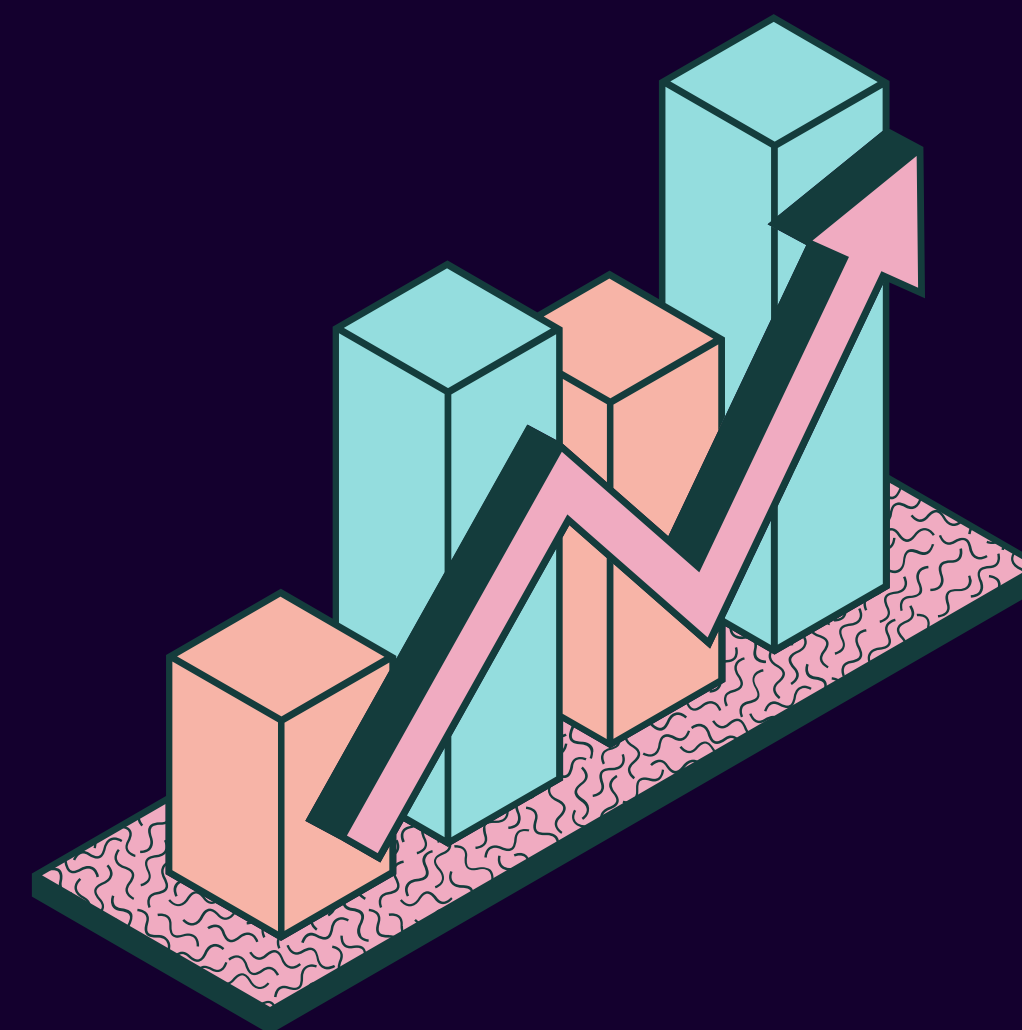




O produto é escalável?

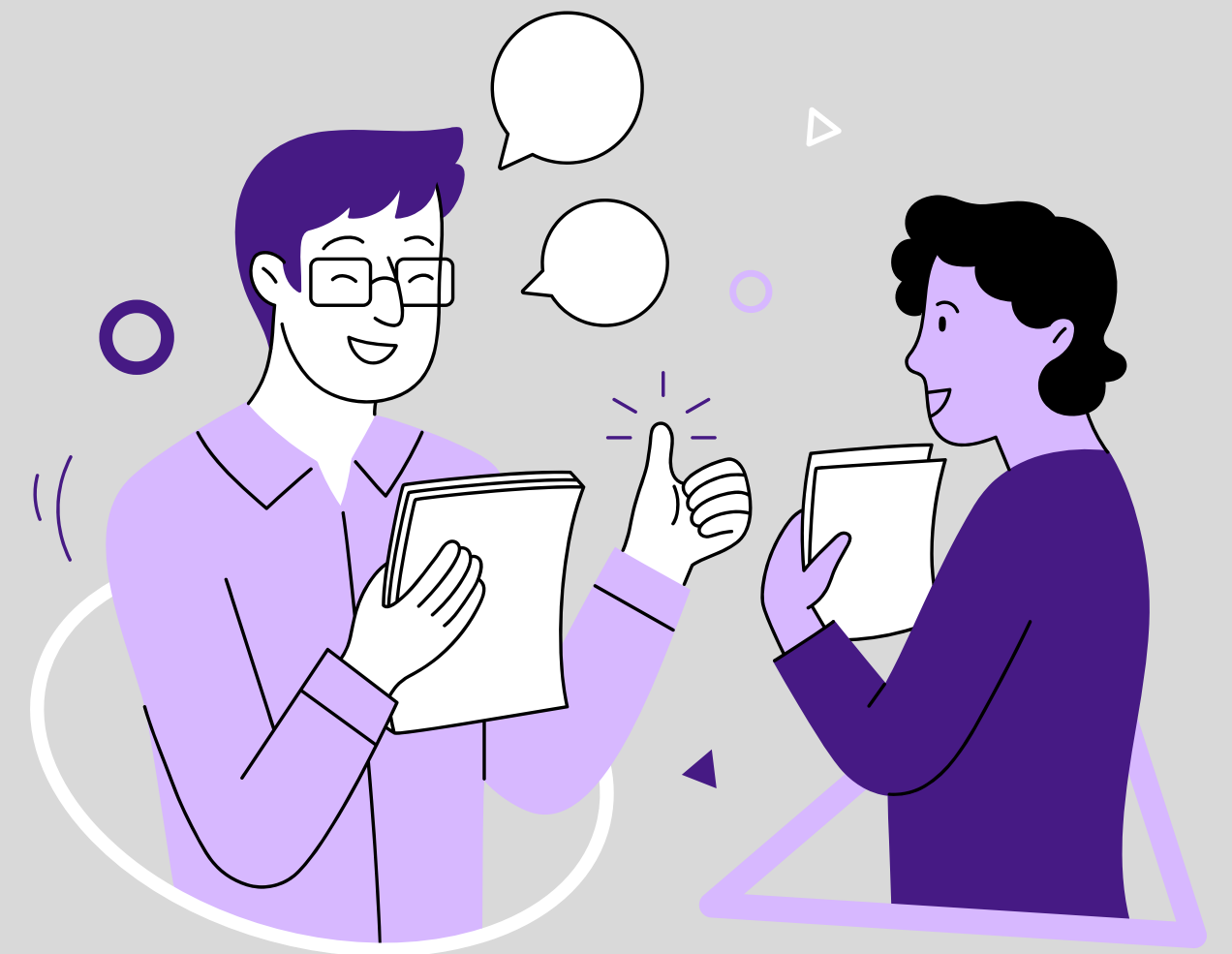


Sim. A escalabilidade do produto é garantida pelo uso do Supabase, que facilita banco de dados, autenticação e armazenamento. Comercialmente, o modelo SaaS e as parcerias B2B permitem crescimento sustentável e novas fontes de receita.



O produto foi utilizado? Houve coleta de feedbacks e melhorias aplicadas?

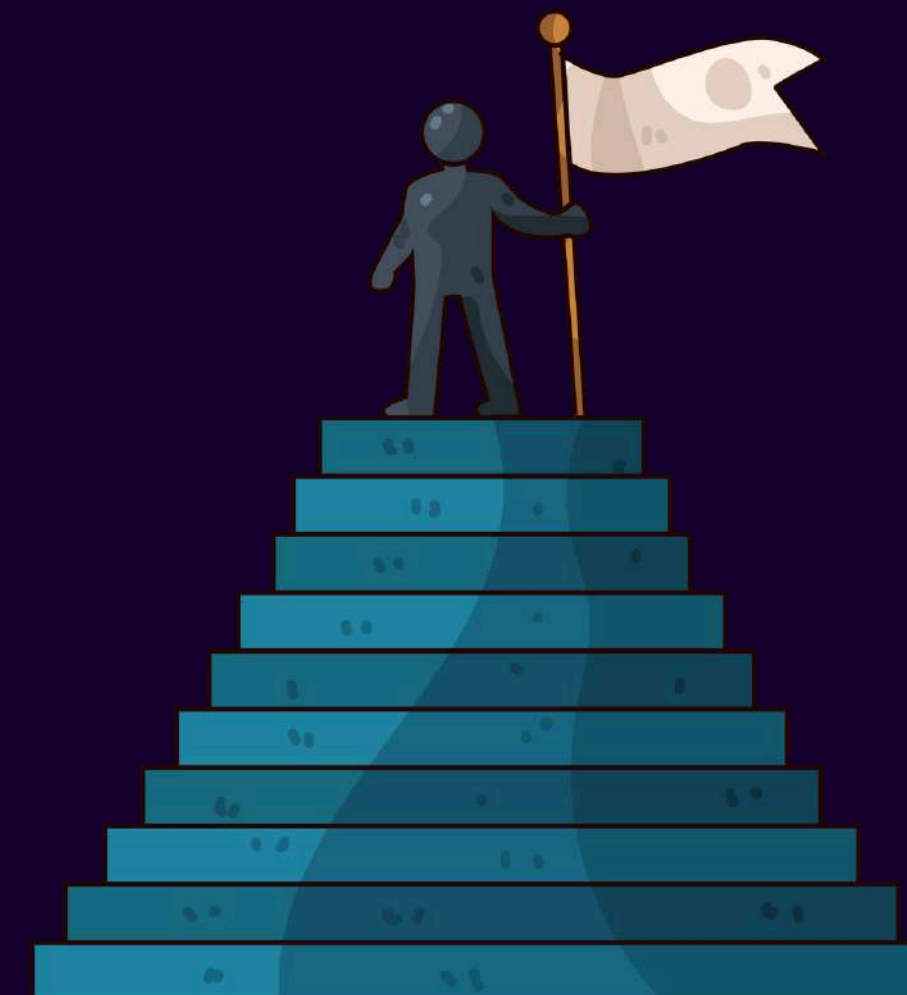
Não houve testes com usuários externos, mas a equipe realizou testagens internas do fluxo de uso, identificando problemas e aplicando melhorias na experiência do usuário e na lógica de algumas funções.





Existe alguma métrica que indica o sucesso do produto?

- Engajamento Social: Número de conexões/amigos criados por usuário.
- Retenção de Cadastro: Taxa de conclusão do onboarding (usuários que preenchem o perfil e salvam hobbies).
- Conversão de Funcionalidades: Quantidade de rotas geradas e o volume de interações com o menu de estimativa de custos.
 - LTV / Churn: Adesão e permanência no modelo de assinaturas.





Houve iteração rápida e adaptação do produto às necessidades e preferências?

Sim. O desenvolvimento evoluiu com adaptações constantes, incluindo melhorias na gestão de sessão por usuário, mais estabilidade e personalização, além de mudanças na UI/UX da tela principal com uma home-bar mais interativa.





Análise dos Critérios Técnicos Fundamentais





Resumo Geral da Avaliação

Critério	Avaliação	Destaque
2a — Processo de Trabalho	✓ Aprovado	Abordagem iterativa e incremental (MVP+)
2b — Gerenciamento e Colaboração	✓ Aprovado	Decisões estratégicas, escopo controlado
2c — Arquitetura de Software	✓ Aprovado	Repository Pattern + BaaS + Event-driven
2d — Usabilidade e UX/UI	✓ Aprovado	Material Design 3, Onboarding, Design contextual
2e — Testabilidade e Manutenibilidade	✓ Aprovado	Estrutura desacoplada, dívida técnica gerenciada
2f — Twelve-Factor App	✓ Aprovado	Aplicada com adaptações ao contexto mobile



Processo de Trabalho do Time

Abordagem adotada

- Desenvolvimento iterativo e incremental
- Projeto evoluiu de MVP → MVP+ com entregas progressivas
- Backlog priorizado e ativo (seção 'Próximos Passos')
- Compatível com metodologias ágeis (Scrum/Kanban)

Evidências no projeto

- Funcionalidades organizadas por módulos coesos
- Critérios claros de conclusão por entrega
- Ciclo completo: requisitos → design → impl. → validação
- Todas as integrações críticas operacionais

Linha do tempo





Processo de Trabalho do Time

Gerenciamento do Projeto

- Decisão estratégica: Supabase (BaaS) para reduzir escopo operacional
- Concentração de esforços no produto, não na infraestrutura
- Backlog gerenciado com clareza de prioridades
- Funcionalidades críticas entregues primeiro (auth, rotas, IA)

Colaboração do Time

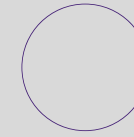
- Amplitude técnica exige perfis complementares
- Android nativo + APIs externas + BaaS + IA generativa
- Coerência arquitetural entre camadas distintas
- Alinhamento técnico evidenciado pela consistência do produto





Arquitetura de Software

Camada	Tecnologia	Papel
Apresentação	Kotlin + XML + Material Design 3	UI e interação com o usuário
Domínio / Dados	Repository Pattern	Abstração e desacoplamento de fontes de dados
Backend (BaaS)	Supabase (PostgreSQL, Auth, Realtime)	Persistência, autenticação, tempo real
IA Generativa	Gemini API	Recomendações personalizadas
Geolocalização	Google Maps & Places SDK	Mapas, busca e dados de locais
Clima	Open-Meteo API	Dados meteorológicos contextuais
Rede / Imagens	Retrofit + OkHttp + Glide	Comunicação HTTP e cache de imagens



Processo de Trabalho do Time

Sistema de Design

- Material Design 3 como base
- Consistência visual e acessibilidade
- Estilo visual exclusivo nos mapas

UX de Entrada

- Onboarding estruturado de preferências
- Busca com Autocomplete
- Filtros rápidos por categoria

Design Contextual

- Pop-ups proativos de sugestão
- Alertas meteorológicos adaptativos
- Informações de suporte à decisão

Resiliência

- Mapas e rotas offline
- Design para baixa conectividade
- Experiência sem degradação





Testabilidade e Manutenibilidade

Testabilidade

- Repository Pattern viabiliza substituição por mocks e fakes
- Pré-requisito fundamental para testes unitários em Android
- Coroutines compatíveis com kotlinx-coroutines-test
- Supabase (BaaS) reduz superfície de testes de integração
- Infraestrutura testada e mantida pelo provedor

Manutenibilidade

- Repository Pattern: troca de fonte de dados sem impacto
- Lifecycle-aware components: zero memory leaks estruturais
- BaaS elimina manutenção de infra pelo time
- Divisão modular por features facilita localização de código
- Refatoração planejada para Jetpack Compose (dívida técnica gerenciada)

Gestão de Dívida Técnica:

A migração gradual para Jetpack Compose está registrada como próximo passo, evidenciando reconhecimento e gerenciamento ativo da dívida técnica — prática de manutenibilidade de alta maturidade.

2f — The Twelve-Factor App Methodology

I. Codebase

Base única e versionada

II. Dependencies

Declaradas via Gradle

III. Config

APIs segregadas do código

IV. Backing Services

Supabase, Gemini, Maps como serviços anexados

V. Build/Release

Pipeline Android com ambientes separados

VI. Processes

App stateless entre sessões

VII. Port Binding

N/A — contexto mobile

VIII. Concurrency

Coroutines para processamento concorrente

IX. Disposability

Lifecycle-aware garante encerramento gracioso

X. Dev/Prod Parity

Supabase oferece ambientes isolados

XI. Logs

Monitoramento integrado ao ciclo de dev

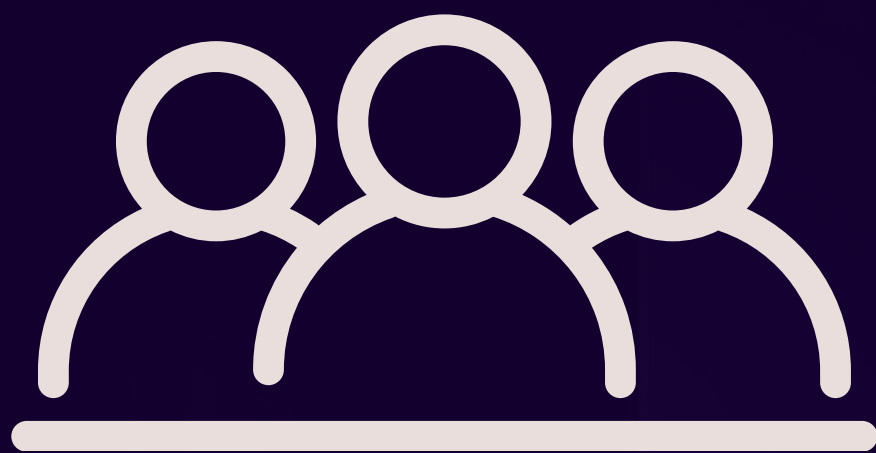
XII. Admin Processes

N/A — gerenciado via console Supabase



Conclusão

- ✓ Todos os 6 critérios técnicos fundamentais foram atendidos
- ✓ Todos os 6 critérios técnicos fundamentais foram atendidos
- ✓ Todos os 6 critérios técnicos fundamentais foram atendidos
- ✓ Todos os 6 critérios técnicos fundamentais foram atendidos
- ✓ Todos os 6 critérios técnicos fundamentais foram atendidos



Equipe do Desenvolvimento

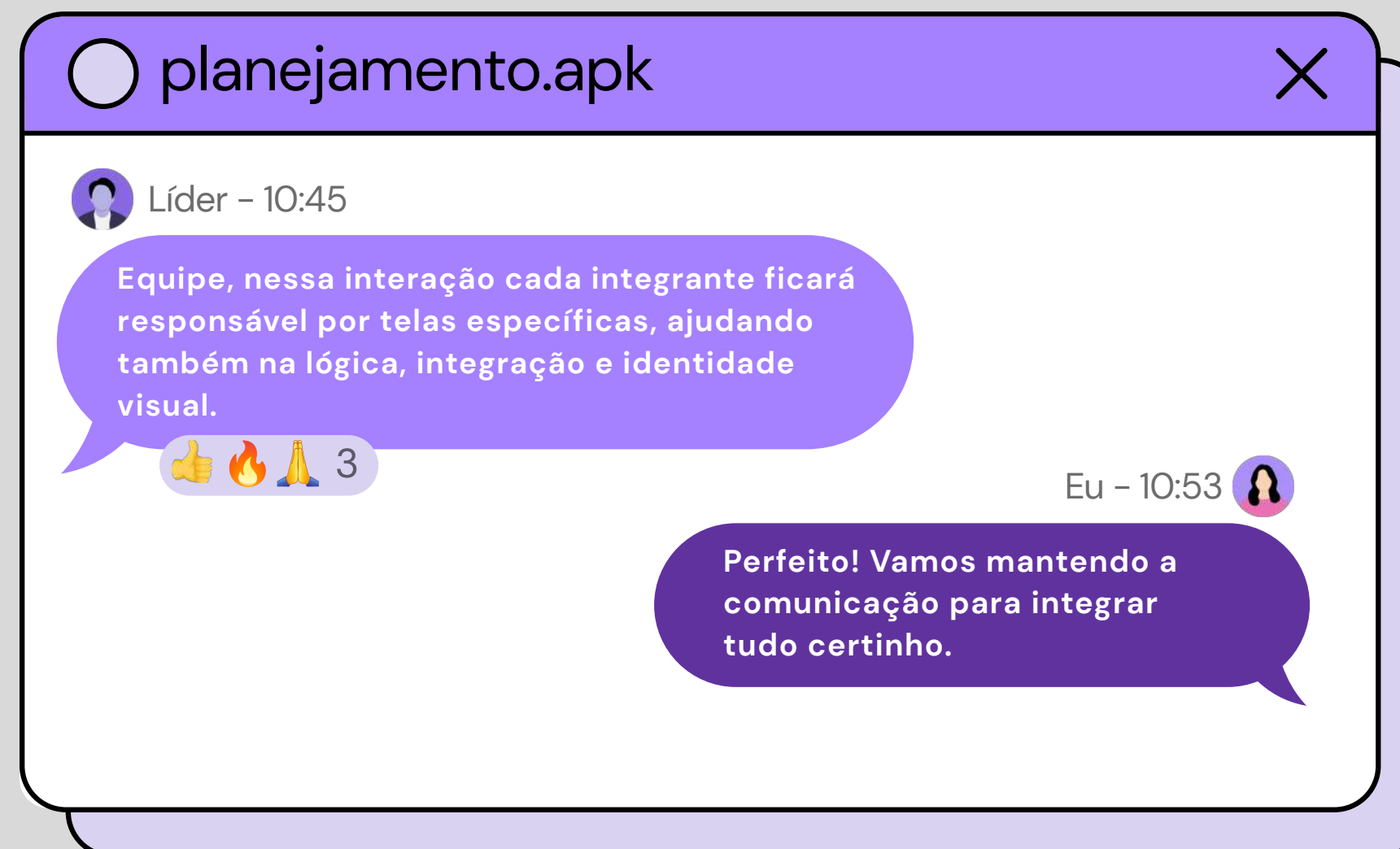
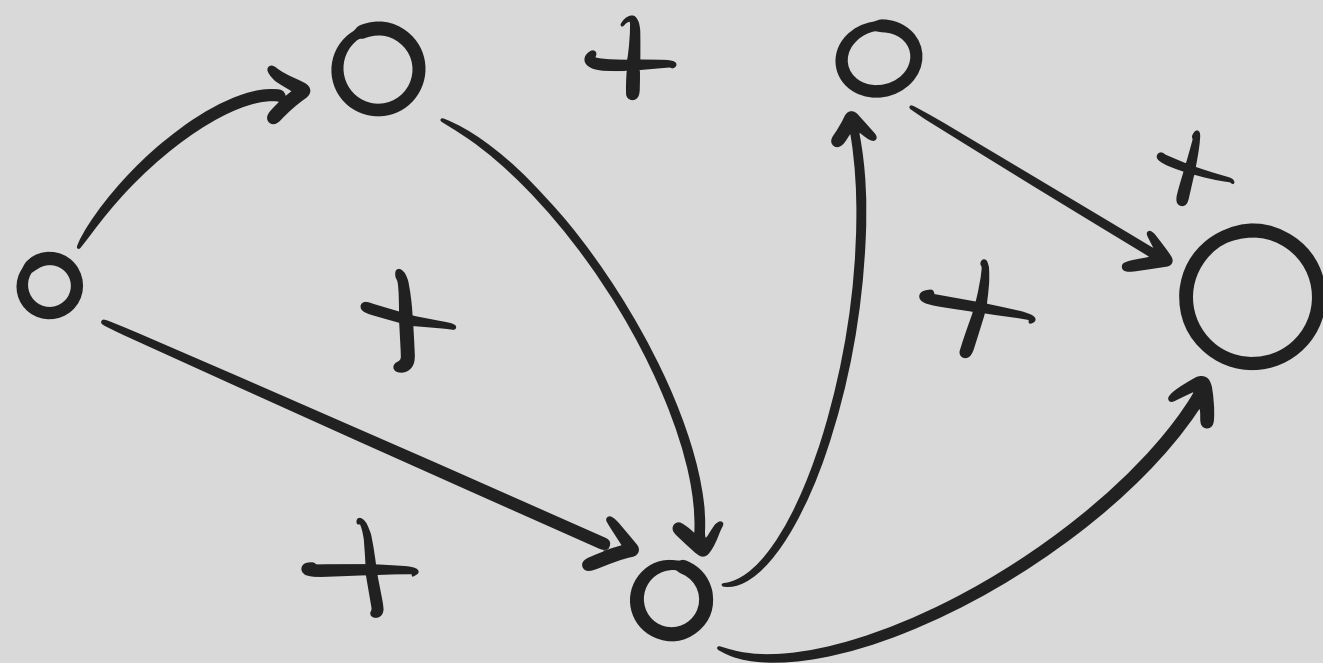
A equipe por trás do OWN MY WAY





Organização da Equipe e Divisão de Tarefas

A equipe atuou de forma colaborativa durante o desenvolvimento do aplicativo, com os integrantes revezando entre tarefas de idealização, front-end, back-end, design e identidade visual.





A organização por sprints foi essencial para acompanhar o progresso do projeto, distribuir melhor as demandas e adaptar as tarefas conforme as necessidades do desenvolvimento. Esse processo permitiu que cada integrante participasse de diferentes etapas, fortalecendo habilidades técnicas, criativas e de trabalho em equipe.

"Segue o rotação de revisões dessa interação."

rotação.txt



Commit realizado

Função de "Esqueci minha senha" implementada.



Pull request aberto

PR#24 enviado. Sua revisão foi solicitada.



Revisão concluída

Pull request aprovado com sucesso.



Membros da equipe

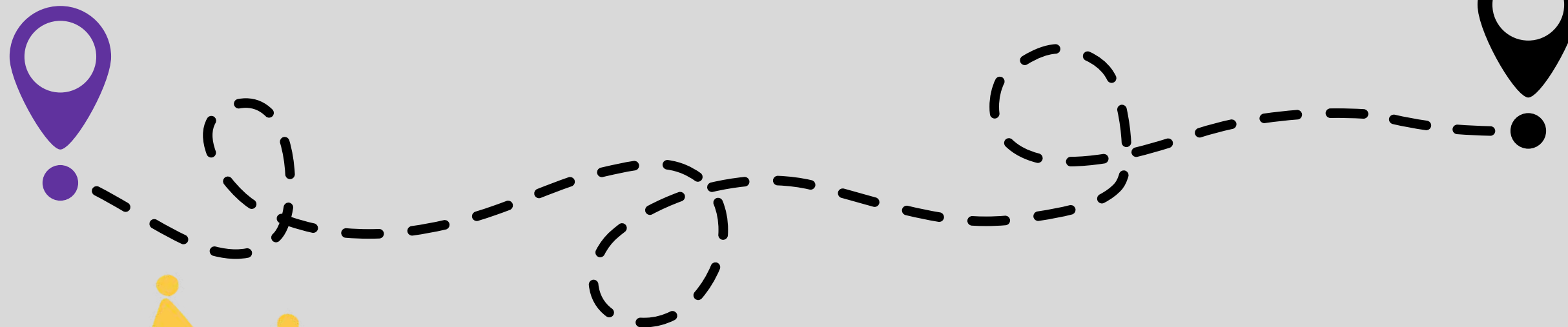


Artur Anderson



Giovana Torres





João Pedro Felix



Lean Albuquerque



Ricardo Rodrigues



Aprendizagem baseada em projetos





Processo ágil aplicado

O aprendizado aconteceu principalmente a partir do desenvolvimento prático do produto. O processo ágil ajudou a organizar as entregas em etapas, permitindo que a equipe testasse, corrigisse e melhorasse o projeto ao longo do tempo.





Auto-gerenciamento das equipes

A equipe desenvolveu autonomia para distribuir responsabilidades e acompanhar o andamento das atividades. Isso contribuiu para que cada integrante entendesse melhor sua função dentro do projeto.





Comunicação e avaliação dos pares

Nos comunicamos via GitHub, em sala de aula e informalmente por WhatsApp. Experiência foi muito boa no geral

A avaliação entre os pares permitiu refletir sobre a participação de cada integrante, reconhecendo contribuições individuais e também pontos que poderiam melhorar no trabalho em equipe.





Sobre a disciplina

A disciplina mostrou a importância do planejamento e da comunicação durante o desenvolvimento de um projeto. Também aprendemos que testar constantemente o produto ajuda a identificar falhas mais cedo. Além de todo o aprendizado na área dos negócios

Positivos

- Liberdade para desenvolvimento e ideias
- Divisão do projeto em partes
- Orientação constante
- Sebrae

Negativos

- Aulas muito longas em meio de sprint
- ?



Obrigada!

